ANNOUNCEMENT

- No class on 11.04 (Wed)
- Checkpoint II presentations will be on 11.09
  - 10-12 min presentation + 3 min Q&A
  - Track I MUST cover:
    - A research problem your team chose
    - Your scientific hypothesis, the approach, and Δ over prior work
    - Your experimental design for evaluating the hypothesis
    - Your preliminary results [very important]
    - Your next steps to complete the research at the final presentation

ANNOUNCEMENT – CONT'D

- No class on 11.04 (Wed)
- Checkpoint II presentations will be on 11.09
  - 10-12 min presentation + 2-3 min Q&A
  - Track 2 MUST cover:
    - The method of your team chose
      - Overview (1 slide)
      - Threat model (1 slide): attacker, defender and what counts as a defense success
      - Method (1-2 slides): input and output; as-is (w/o defense) vs. to-be (with defense)
    - Reproduction target
      - The paper's results: numbers, metrics, models and datasets
      - Your success criterion: how you will judge the reproduction?
    - Preliminary results [required]
      - Your results and comparison to the paper's result
    - Next steps: what remains until the final presentation and who does it

CS 578: CYBER-SECURITY

PART III: SIDE-CHANNELS

Sanghyun Hong
sanghyun.hong@oregonstate.edu





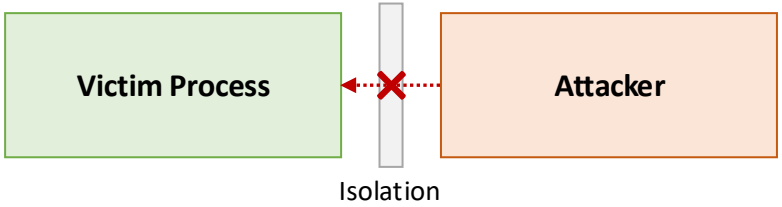
RECAP – WHERE WE ARE

- Part I – Network/Internet security
  - The attacker is *outside* your machine
- Part II – Computer systems security
  - The attacker is *inside* your machine
  - Memory safety: *how* the attacker gets in
  - OS security: *decide what* the attacker may do (access control)
- Part III – Isolation and (software-induced) breaks
  - The attacker is *inside your program* or is *the OS* itself
  - Isolate untrusted code in your program (SFI) and yours from untrusted OS (Haven)
  - The attacker exploits *your hardware* and break *integrity and confidentiality* *(today!)*

SIDE-CHANNEL ATTACKS – EXPLOITING INFORMATION LEAKAGE

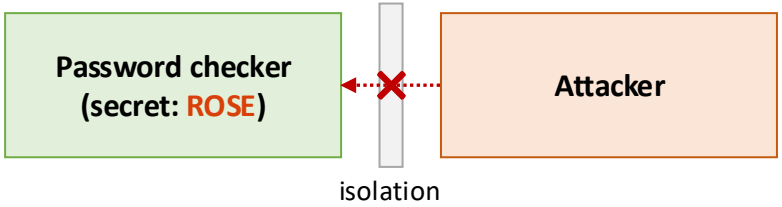
REVISIT THE ISOLATION

- Goal
  - Protect memory *integrity* and *confidentiality*
  - SFI and Haven protect the integrity



REVISIT THE ISOLATION – CONT'D

- Goal
  - Protect memory *integrity* and *confidentiality*
  - SFI and Haven protect the integrity

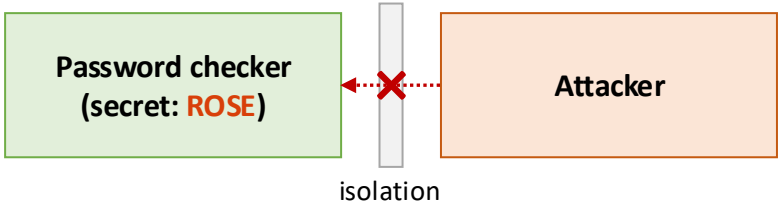


- Password checker example
  - Suppose it checks each char from the left
  - The check stops at the first wrong character

Guess	Answer	Response time
AAAA	wrong	

REVISIT THE ISOLATION – CONT'D

- Goal
  - Protect memory *integrity* and *confidentiality*
  - SFI and Haven protect the integrity



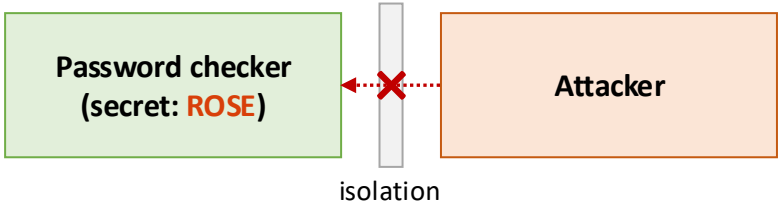
- Password checker example
  - Suppose it checks each char from the left
  - The check stops at the first wrong character
  - Time tells how many chars we were correct

Guess	Answer	Response time
AAAA	wrong	
RAAA	wrong	
ROAA	wrong	

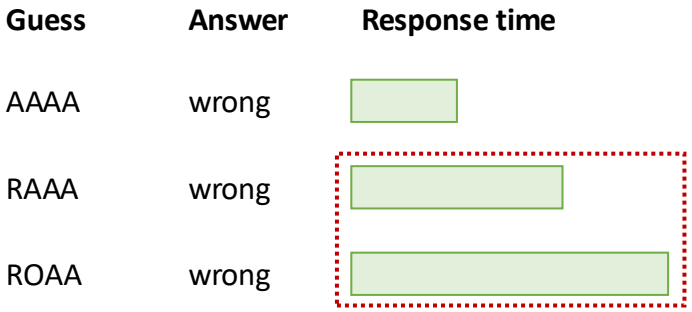
RAAA and ROAA take longer: R, then O, is right

REVISIT THE ISOLATION – CONT'D

- Goal
  - Protect memory *integrity* and *confidentiality*
  - SFI and Haven protect the integrity



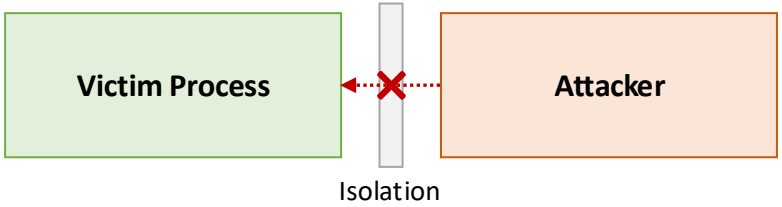
- Password checker example
  - Suppose it checks each char from the left
  - The check stops at the first wrong character
  - Time tells how many chars we were correct
  - ex) 26 char x 4 letters = 104 guesses



MICRO-ARCHITECTURAL SIDE-CHANNEL ATTACKS

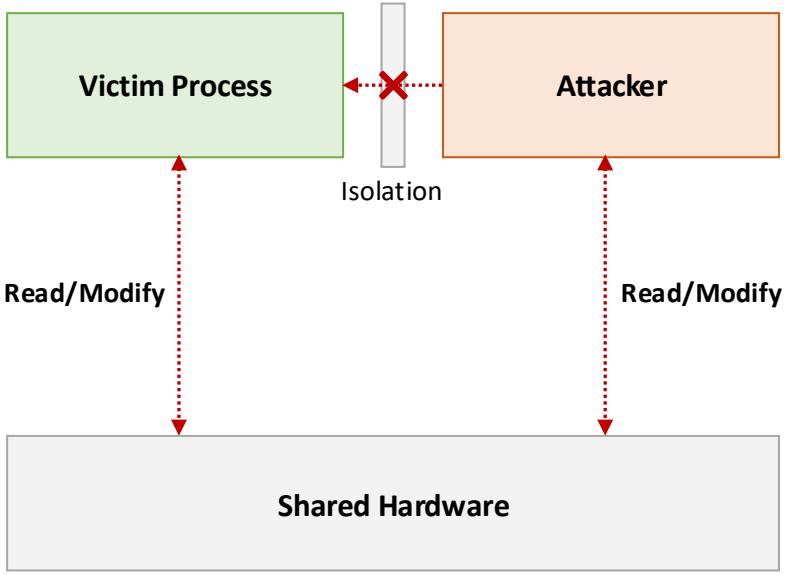
TWO ISOLATED PROCESSES

- Password checker example
  - Time tells how many chars we were correct
  - Solution: fix code not to leak the timings (code exec. should not depend on a secret)



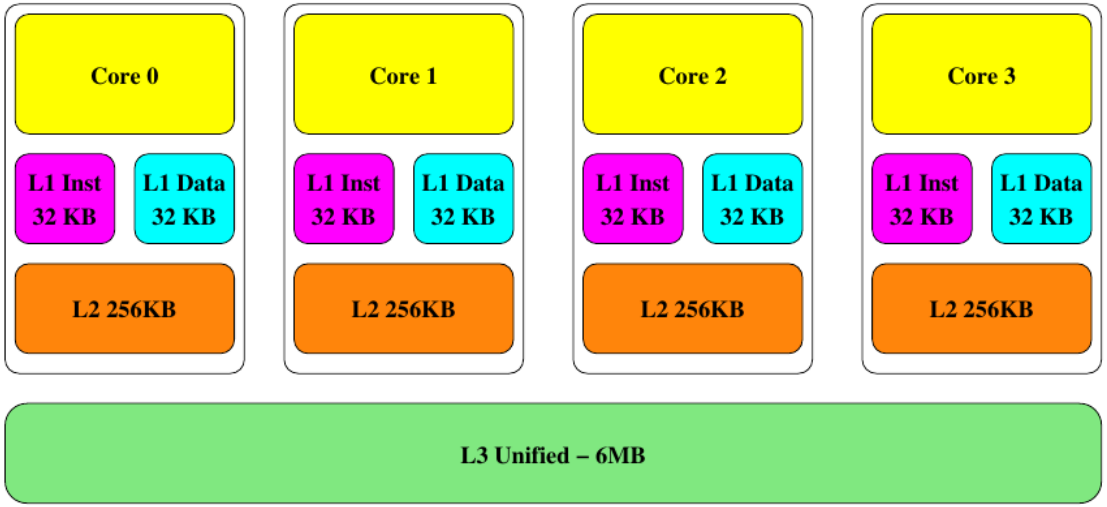
TWO ISOLATED PROCESSES SHARE THE SAME HARDWARE STATE

- Password checker example
  - Time tells how many chars we were correct
  - Solution: fix code not to leak the timings (code exec. should not depend on a secret)
- The code runs on the same host
  - Code alters the host hardware states
  - The attacker can observe such state changes



WHAT IS HARDWARE (MICRO-ARCHITECTURAL) STATE SHARED

- **Cache**
  - Per-core: L1 (32KB + 32KB)
  - Per-core: L2 (256KB)
  - Shared : L3 (6MB)
    (The last-level cache, LLC)



- **Unit of data transfers**
  - A cache line (64 bytes)

- **Access time**
  - Registers > L1 > L2 > L3 >>>>> DRAM

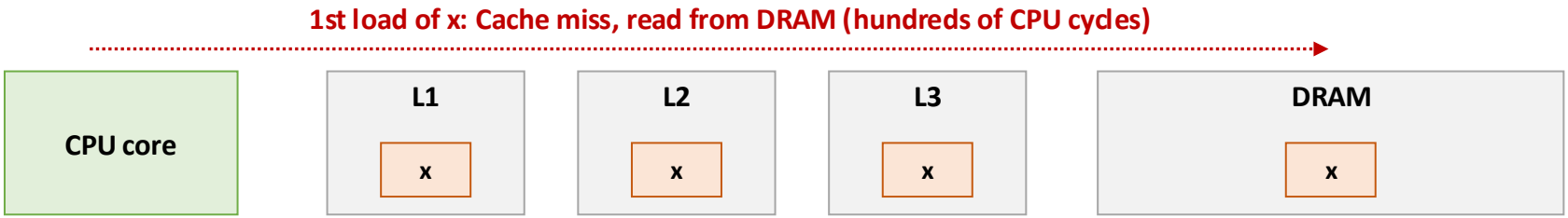
CACHE ACCESS TIMING

- Difference in data access time



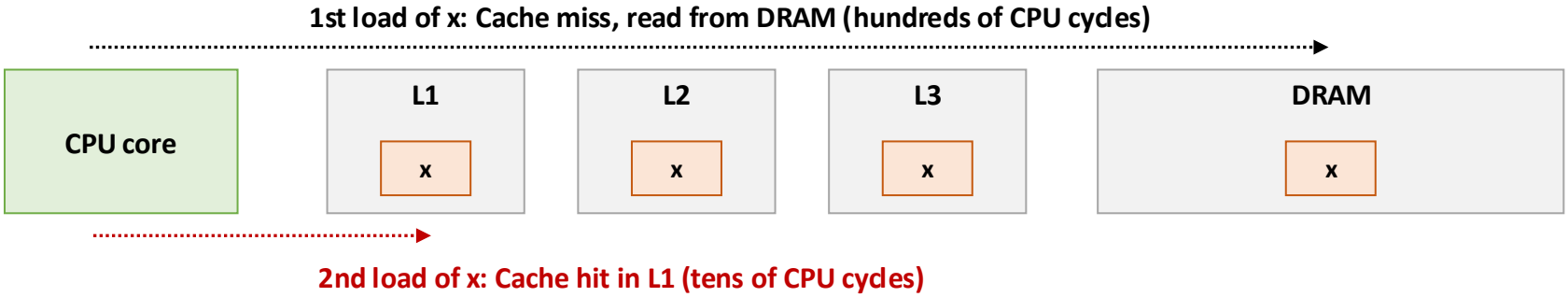
CACHE ACCESS TIMING – CONT'D

- Difference in data access time
  - Miss (slow): the data is NOT in cache



CACHE ACCESS TIMING – CONT'D

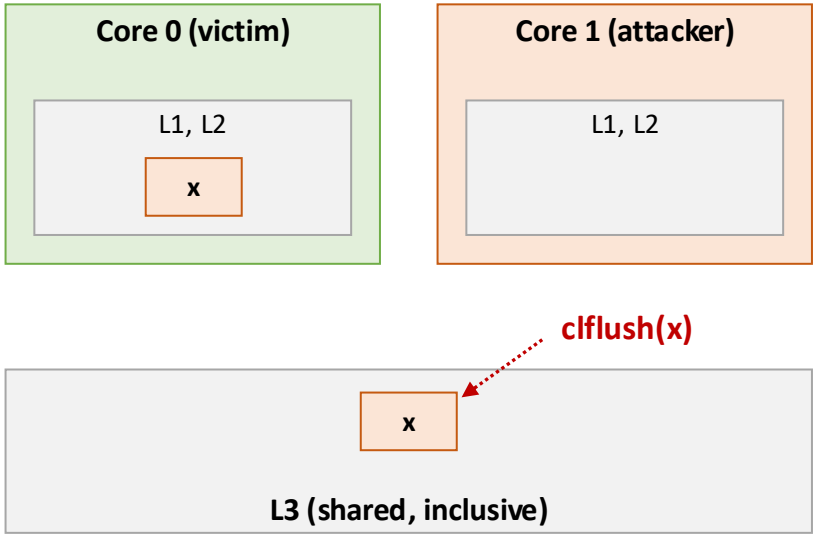
- Difference in data access time
  - Miss (slow): the data is NOT in cache
  - Hit (fast): the data in cache (e.g., x is now in every cache level)



FLUSH+RELOAD – L3 CACHE SIDE-CHANNEL ATTACK

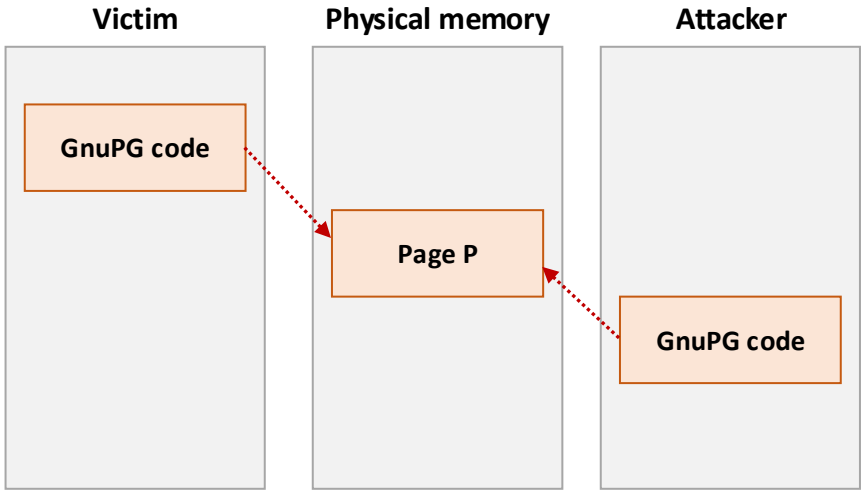
PRELIM – INCLUSIVE CACHE AND CLFLUSH

- Intel's LLC is *inclusive*
  - L3 holds a copy of every line in L1 and L2
- clflush instruction
  - Evict one cache line from all levels
  - No permission checks on x86



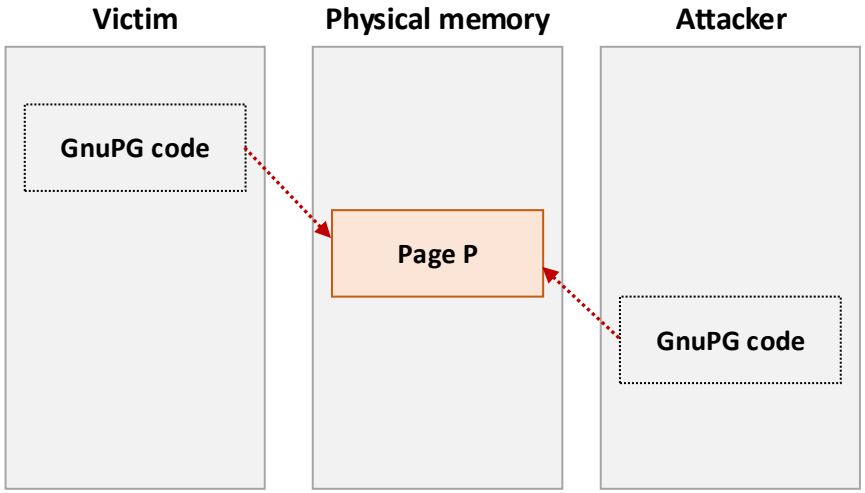
PRELIM – PAGE SHARING

- OS mechanism
  - To share identical pages to save memory
  - Useful for shared libs and executables



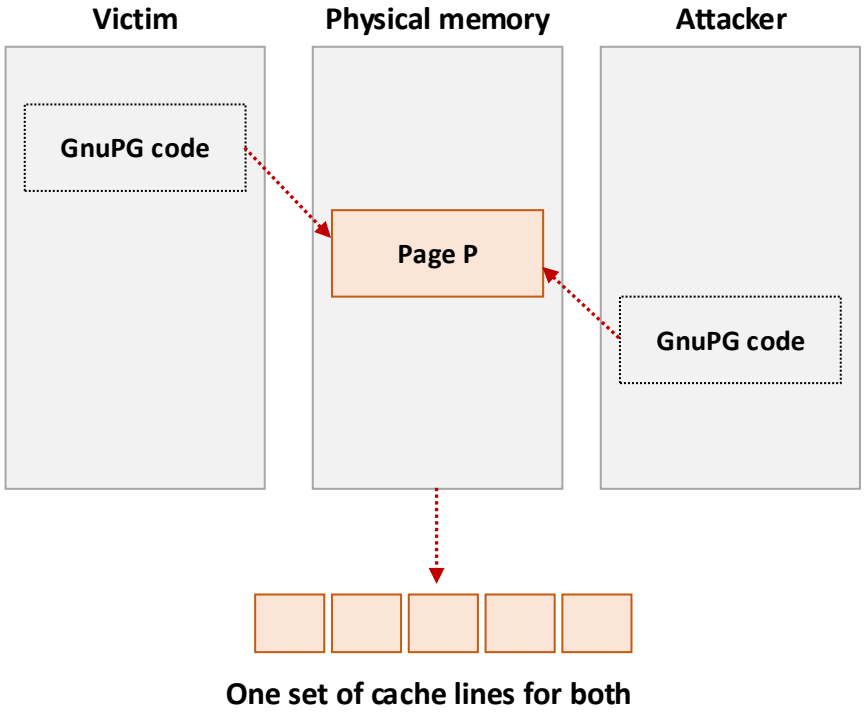
PRELIM – PAGE SHARING

- OS mechanism
  - To share identical pages to save memory
  - Useful for shared libs and executables
  - De-duplication: VMware ESX, KSM



PRELIM – PAGE SHARING

- OS mechanism
  - To share identical pages to save memory
  - Useful for shared libs and executables
  - De-duplication: VMware ESX, KSM
- Copy-on-write
  - No one can modify the P
  - Both attacker and victim use the same P
  - And so the same cache lines



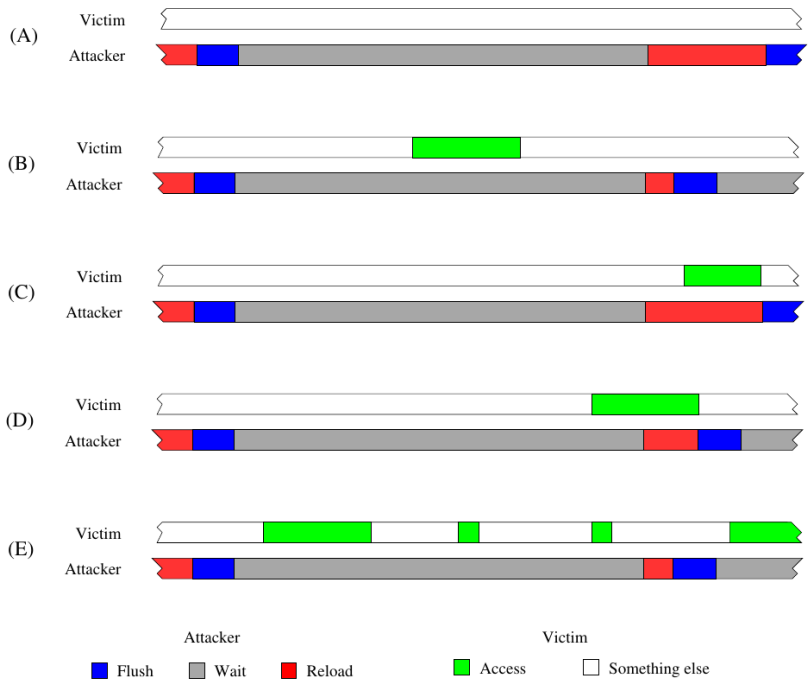
FLUSH+RELOAD – CACHE SIDE-CHANNEL ATTACK

- How it works
  - Both attacker and victim shares P
  - The attacker clflushes a shared line
  - The attacker waits for a while
  - The attacker reloads it and times the load
  - Repeat the previous three steps

FLUSH+RELOAD – CACHE SIDE-CHANNEL ATTACK

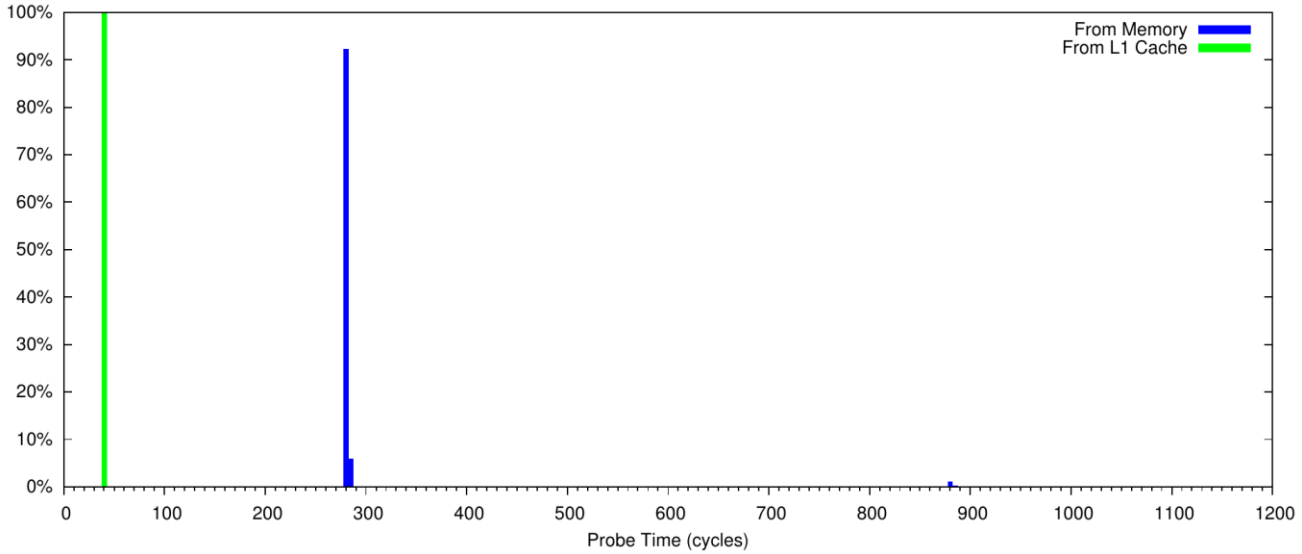
- How it works
  - Both attacker and victim shares P
  - The attacker clflushes a shared line
  - The attacker waits for a while
  - The attacker reloads it and times the load
  - Repeat the previous three steps

- Possible consequences
  - The victim accessed the flushed line (fast ld)
  - The victim did not access the line (slow ld)
  - It could be noisy in reality; needs some work



FLUSH+RELOAD – HOW NOISY THE SIGNAL IS

- Compare the time it takes to read from the cache and memory
  - From cache, it takes < 100 ms
  - From memory, it could take ~300ms
  - The attacker can set the threshold to ~150ms, then they can distinguish the victim's access



FLUSH+RELOAD – THREAT MODEL

- Scenario
  - The attacker and victim shares the same host machine (but not running on the same core)
  - The OS uses the page sharing mechanism (maintains the copy with de-duplication)
  - The victim's memory accesses depend on the secret
- Knowledge
  - The attacker knows the victim's code and layout (both use the shared libraries)
  - The victim does not know the attacker is monitoring the access
- Capabilities
  - The attacker can use the OS mechanism to share the same page(s)
  - The attacker can run unprivileged code (e.g., clflush)
  - The attacker can measure a (not that) precise cache access time

ATTACKING GNUPG – LEAKS

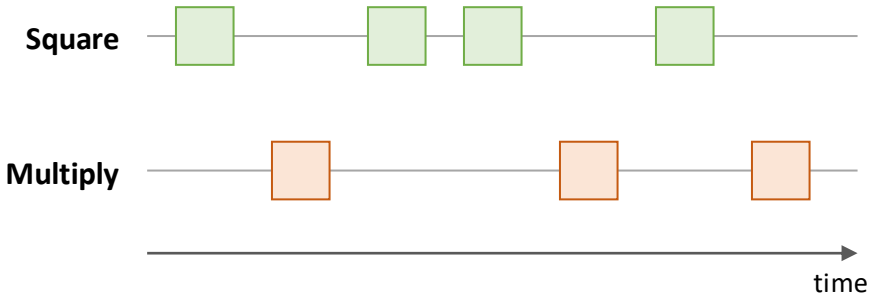
- Crypto code as an attack target
  - The attacker may extract the private key
  - It can be used for downstream attacks
- RSA
  - Compute c^d mod n (d is the private key)
  - Square-n-multiply scans the bits of d
  - Bit = 0: square, reduce
  - Bit = 1: square, reduce, multiply, reduce
- Code execution depends on d

```
1  function exponent(b, e, m)
2  begin
3    x ← 1
4    for i ← |e| − 1 downto 0 do
5      x ← x^2
6      x ← x mod m
7      if (e_i = 1) then
8        x ← xb
9        x ← x mod m
10     endif
11   done
12   return x
13 end
```

ATTACKING GNUPG – RECOVERING D

- Example: d = 1011 (binary)
  - The spy watches the square code and the multiply code

Bit of d	The victim runs
1	square, multiply
0	square
1	square, multiply
1	square, multiply



S M · S · S M · S M → d = 1 0 1 1

ATTACKING GNUPG – RESULTS

- Scenario
  - RSA in GnuPG 1.4.13
  - Co-location: two processes in one OS; two VMs on one host
  - One signing (or decryption round)

- Results
  - The same OS: 98.7% of the key bits on average
  - The same host: 96.7% across VMs
  - 2 rounds were sufficient to recover the whole key

- Disclosed to the developers before publication

FLUSH+RELOAD – COUNTERMEASURES

- Potential countermeasures

Capability required	Countermeasure	Problem
Unprivileged clflush	Disable unprivileged clflush	New CPU instruction needed
De-duplication Page sharing	Disable de-duplication (Most cloud provide did this)	More memory required
Precise timer (~10ms granularity)	Coarse clock, randomize	The attacker can write their own clock (cntering thread)
Secret dependent code	Constant time operations	Slow; only prevent this attack

SPECTRE ATTACKS – EXPLOITING SPECULATIVE EXECUTION
IN-CLASS PRESENTATION BY JARED AND SATVIK

PRELIM – INSTRUCTION PIPELINE

- Instruction cycle
  - Fetch
  - Decode
  - Execute
  - Memory
  - Write back

- Instruction *pipelining*
  - Start the next instruction before one ends
  - At clock 4, four instructions are in flight

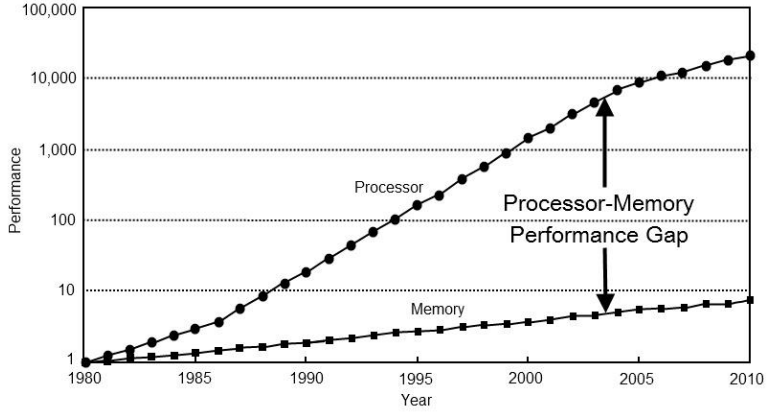
Instr. No.	Pipeline Stage						
1	IF	ID	EX	MEM	WB		
2		IF	ID	EX	MEM	WB	
3			IF	ID	EX	MEM	WB
4				IF	ID	EX	MEM
5					IF	ID	EX
Clock Cycle	1	2	3	4	5	6	7

PRELIM – SPECULATIVE EXECUTION

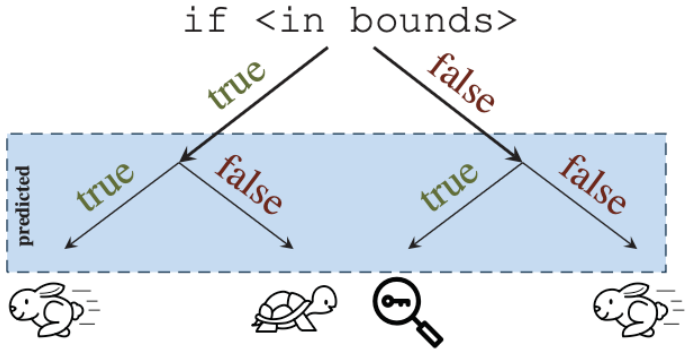
- Problem
  - An instruction that we cannot pipeline

```
if (x < array1_size)
    y = array2[array1[x] * 4096];
```

  - Suppose “array1_size” is in DRAM
  - CPU needs to wait hundreds of cycles there

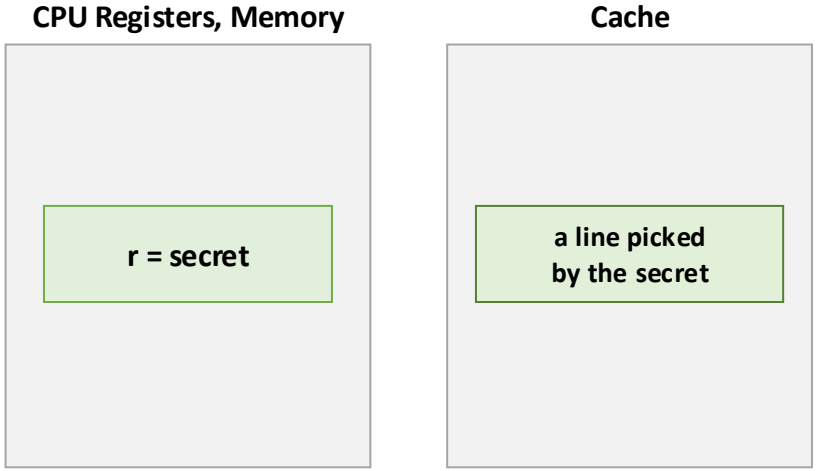


- Speculative execution
  - Based on the branch history,
    CPU guesses the branch and keeps running
  - If the guess is correct later on, commit the results
  - If the guess is wrong, throw them away



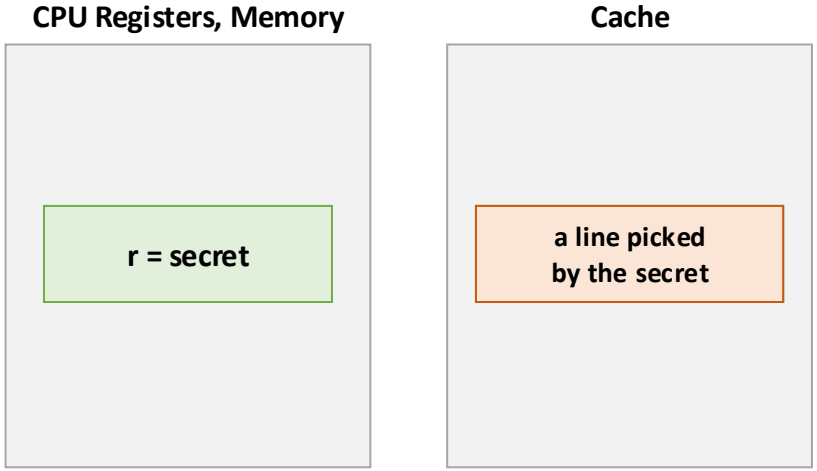
PRELIM – AN INCORRECT GUESS LEAVES THE TRACES IN MEMORY

- During speculative execution
  - CPU predicts a branch and loads data
  - The data is in registers, cache and memory



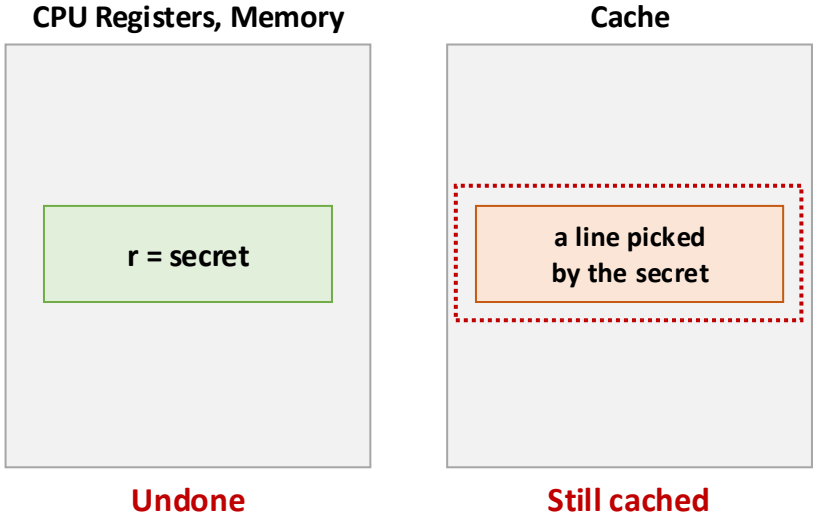
PRELIM – AN INCORRECT GUESS LEAVES THE TRACES IN MEMORY

- During speculative execution
  - CPU predicts a branch and loads data
  - The data is in registers, cache and memory
  - Unfortunately, the prediction was incorrect



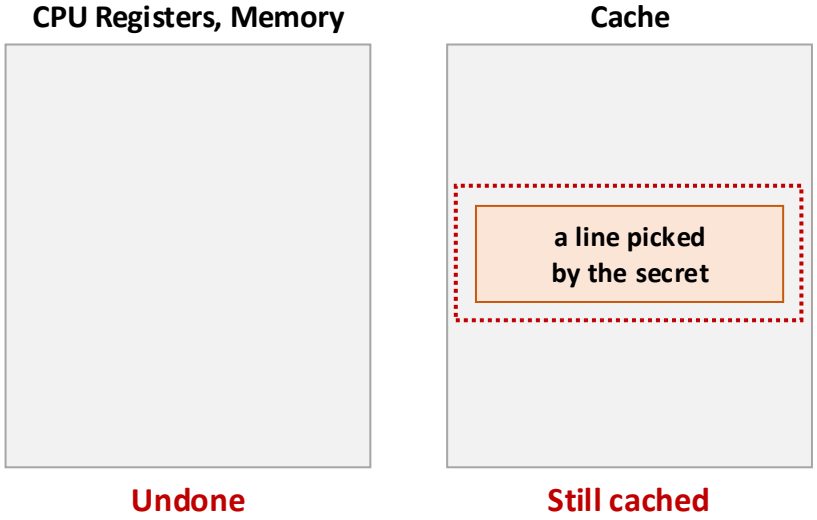
PRELIM – AN INCORRECT GUESS LEAVES THE TRACES IN MEMORY

- During speculative execution
  - CPU predicts a branch and loads data
  - The data is in registers, cache and memory
  - Unfortunately, the prediction was incorrect
  - CPU runs a *rollback*
    - Registers and memory return to old data
    - But the cache keeps the loaded lines



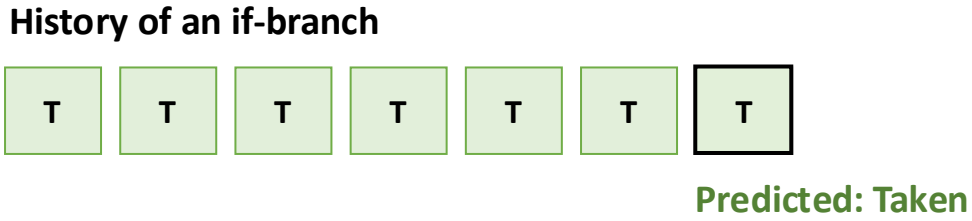
PRELIM – AN INCORRECT GUESS LEAVES THE TRACES IN MEMORY

- During speculative execution
  - CPU predicts a branch and loads data
  - The data is in registers, cache and memory
  - Unfortunately, the prediction was incorrect
  - CPU runs a *rollback*
    - Registers and memory return to old data
    - But the cache keeps the loaded lines
- Side-channel attacker
  - Can read these cache lines
  - Restore the secret(s) removed



PRELIM – BRANCH PREDICTOR

- Programs branch often
  - A program branches once every 5-7 instructions
  - CPU predicts from this branch history



PRELIM – BRANCH PREDICTOR

- Programs branch often
  - A program branches once every 5-7 instructions
  - CPU predicts from this branch history

- Suppose an adversary compromises the history
  - The attacker can call the code with invalid inputs many times
  - The victim's code speculatively branches to the code the attacker wants

History of an if-branch

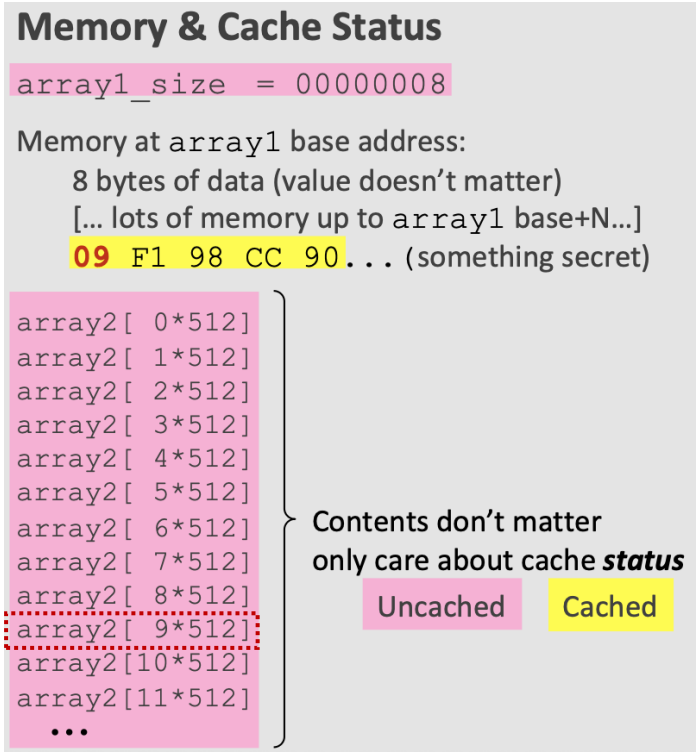


Predicted: branch an attacker wants

SPECTRE – VARIANT 1, BOUNDS CHECK BYPASS

```
if (x < array1_size)
    y = array2[array1[x] * 4096];
```

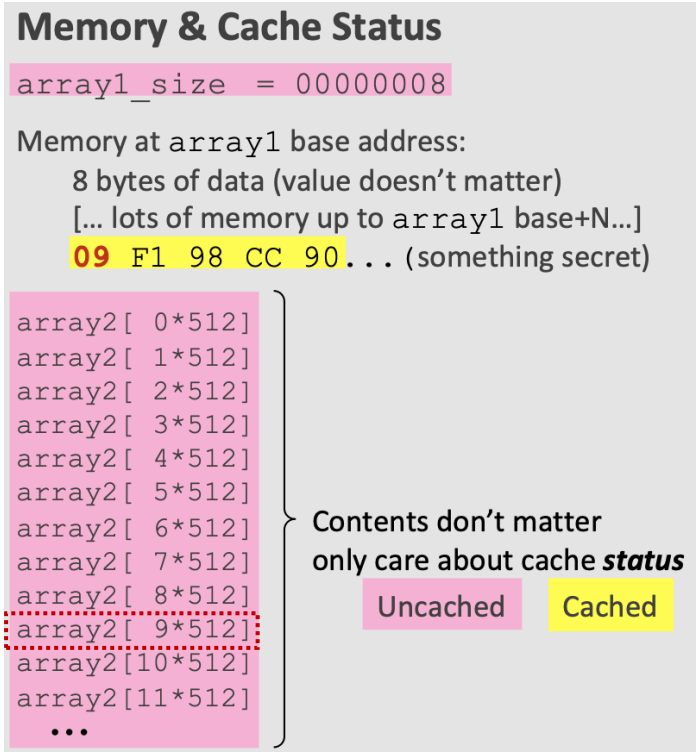
- Attack procedure
  - Call it with valid x many times (train)
  - Set x points at the secret
  - Make sure that array1_size is not cached yet
- Speculative execution
  - Now the CPU guesses x is in bounds
  - Reads the secret byte (09)
  - Loads array2[09 × 4096] into the cache



SPECTRE – VARIANT 1, BOUNDS CHECK BYPASS

```
if (x < array1_size)
    y = array2[array1[x] * 4096];
```

- Attack procedure
  - Call it with valid x many times (train)
  - Set x points at the secret
  - Make sure that array1_size is not cached yet
- Speculative execution
  - Now the CPU guesses x is in bounds
  - Reads the secret byte (09)
  - Loads array2[09 × 4096] into the cache
- Flush+reload over array2: only line 09 is fast!

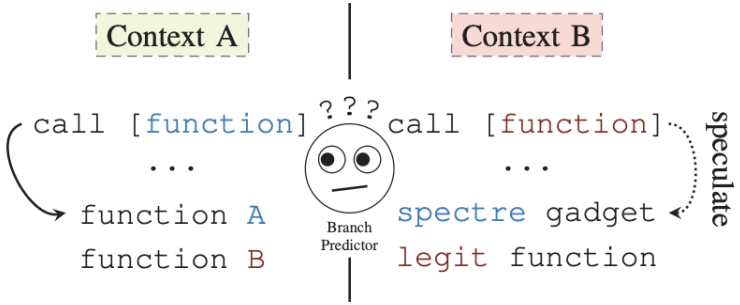


SPECTRE – VARIANT 2, BRANCH TARGET INJECTION

- Preliminaries
  - Indirect branch: jump to an addr in a register
  - Used for function pointers, virtual calls

SPECTRE – VARIANT 2, BRANCH TARGET INJECTION

- Preliminaries
  - Indirect branch: jump to an addr in a register
  - Used for function pointers, virtual calls
- Attack procedure
  - Compromise the branch predictor (train)
  - The predictor did not load the address
  - But it guesses the target from earlier jumps
  - The victim (B) speculatively jumps to a gadget
  - The gadget leaks a secret into the cache (load)
  - The story goes the same as the variant 1 (FR)



RECAP – WHERE WE ARE

- Part I – Network/Internet security
  - The attacker is *outside* your machine
- Part II – Computer systems security
  - The attacker is *inside* your machine
  - Memory safety: *how* the attacker gets in
  - OS security: *decide what* the attacker may do (access control)
- Part III – Isolation and (software-induced) breaks
  - The attacker is *inside your program* or is *the OS* itself
  - Isolate untrusted code in your program (SFI) and yours from untrusted OS (Haven)
  - The attacker exploits *your hardware* and break *integrity and confidentiality*

Thank You!

Sanghyun Hong

https://true-lab.ai/courses/Sec-Grad/current



